NUMPY

NumPy is a Python library.

NumPy is used for working with arrays.

NumPy is short for "Numerical Python".

Python Lists	Python Arrays
Collects items of multiple data types	Collects items of same data type
Uses more memory	Uses less memory
Flexible, Easy to modify	Not fexible, Difficult to modify
No need for a specific loop to access the components	Needs a loop to access the components of array
Good for shorter sequence of data	Good for longer sequence of data

In [30]:
```
x=[1,2,3,"april",1.67]
x[3]
```

Out[30]:
```
'april'
```

importing numpy

In [33]:
```
import numpy as np
np.array([1,2,3,4,5])
```

Out[33]:
```
array([1, 2, 3, 4, 5])
```

creating an array

In [36]:
```
#Create an alias with the as keyword while importing:
import numpy  as np

arr = np.array([1, 2, 3, 4, 5])

print(arr)
arr
```

Out[36]:
```
[1 2 3 4 5]
array([1, 2, 3, 4, 5])
```

In [3]:
```
type(arr)
```

Out[3]:
```
numpy.ndarray
```

In [4]:
```
import numpy as np

arr = np.array((1, 2, 3, 4, 5))

print(arr)
```

```
[1 2 3 4 5]
```

zero -diamension array

0-D arrays, or Scalars, are the elements in an array. Each value in an array is a 0-D array.

In [5]:
```
import numpy as np

arr = np.array(42)

print(arr)
```

```
42
```

In [6]:
```
arr.shape   #Notice that the output is an empty tuple (), indicating that the array has zero dimensions.
```

Out[6]:
```
()
```

one diamention array

An array that has 0-D arrays as its elements is called uni-dimensional or 1-D array.

In [39]:
```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])

print(arr)
print(arr.shape)
```

```
[1 2 3 4 5]
(5,)
```

In [ ]:
```
(r,g,b) #red, green ,blue (0 - 255)


#(0,0,0)
#(255,255,255)
#(0,100,0)
```

two diamention array

An array that has 1-D arrays as its elements is called a 2-D array.

These are often used to represent matrix or 2nd order tensors.

In [40]:
```
np.array([[1,2],[3,5]  ])
```

Out[40]:
```
array([[1, 2],
       [3, 5]])
```

In [8]:
```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]])

print(arr)

arr.shape
```

Out[8]:
```
[[1 2 3]
 [4 5 6]]
(2, 3)
```

(2, 3) indicates that the array has 2 rows and 3 columns.

three diamention

In [45]:
```
x=np.array([[[1,2,3,3],[3,4,4,1]],[[11,20,3,4],[35,14,4,5]]])
x
```

Out[45]:
```
array([[[ 1,  2,  3,  3],
        [ 3,  4,  4,  1]],

       [[11, 20,  3,  4],
        [35, 14,  4,  5]]])
```

In [49]:
```
x.shape # panel, row, col
```

Out[49]:
```
(2, 2, 4)
```

In [9]:
```python
import numpy as np

# Create a 3D array
arr_3d = np.array([[[1, 2, 3], [4, 5, 6]], [[7, 8, 9], [10, 11, 12]]])

# Get the shape
dimensions = arr_3d.shape

print("Dimensions of the 3D array:", dimensions)
```

```
Dimensions of the 3D array: (2, 2, 3)
```

(2, 2, 3) indicates that the array has 2 matrices, each containing 2 rows and 3 columns.

Access Array Elements

Array indexing is the same as accessing an array element.

You can access an array element by referring to its index number.

The indexes in NumPy arrays start with 0, meaning that the first element has index 0, and the second has index 1 etc.

In [52]:
```python
import numpy as np

arr = np.array([1, 2, 3, 4])

print(arr[0])

arr[-1]
```

Out[52]:
```
1
4
```

In [54]:
```python
import numpy as np

arr = np.array([1, 2, 3, 4])

print(arr[2] + arr[3]) # 3+4
```

```
7
```

Access 2-D Arrays

In [56]:
```
import numpy as np

arr = np.array([[1,2,3,4,5], [6,7,8,9,10]])

arr
```

Out[56]:
```
array([[ 1,  2,  3,  4,  5],
       [ 6,  7,  8,  9, 10]])
```

In [61]:
```
arr[1,-1]
```

Out[61]:
```
10
```

In [58]:
```
#first row, second element

print('2nd element on 1st row: ', arr[0, 1])  #R=0 C=1
```

```
2nd element on 1st row:  2
```

?Access the element on the 2nd row, 5th column:

In [14]:
```
print('5th element on 2nd row: ', arr[1, 4])
```

```
5th element on 2nd row:  10
```

In [15]:
```
print('Last element from 2nd dim: ', arr[1, -1])
```

```
Last element from 2nd dim:  10
```

Checking the Data Type of an Array

In [16]:
```
import numpy as np

arr = np.array([1, 2, 3, 4, 8])

print(arr.dtype)  #int32: Represents 32-bit signed integers. It occupies 32 bits (or 4 bytes) of memory.
```

```
int32
```

The first is that python integers are flexible-sized - int

The numpy integers, on the other hand, are fixed-sized. This means there is a maximum value they can hold. This is defined by the number of bytes in the integer (int32 vs. int64),

Another advantage of fixed-sized values is that they can be placed into consistently-sized adjacent memory blocks of the same type. This is the format that numpy arrays use to store data.

In [17]:
```
import numpy as np

# Create a NumPy array with dtype int64
arr = np.array([5], dtype='int64')

print(arr.dtype)  # Output will be int64
```

```
int64
```

In [62]:
```
arr = np.array([3.14, 2.718, 1.618], dtype=np.float64)
print(arr.dtype)
```

```
float64
```

In [19]:
```
arr = np.array([5, 'banana', True], dtype=object)
print(arr.dtype)
```

```
object
```

In [20]:
```
arr = np.array(['apple', 'banana', 'cherry'], dtype=object)
print(arr.dtype)
```

```
object
```

NumPy Array Reshaping

Reshaping means changing the shape of an array.

The shape of an array is the number of elements in each dimension.

By reshaping we can add or remove dimensions or change number of elements in each dimension.

In [68]:
```
import numpy as np
```

```
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11,12])

newarr = arr.reshape(4, 3)

print(newarr)
```

```
[[ 1  2  3]
 [ 4  5  6]
 [ 7  8  9]
 [10 11 12]]
```

In [22]:
```
newarr = arr.reshape(3,4)

print(newarr)
```

```
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
```

? why error

In [69]:
```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7, 8,9])

newarr = arr.reshape(3, 3)

print(newarr)
```

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

convert 2D into 1D

In [24]:
```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]])

newarr = arr.reshape(-1)

print(newarr)
```

```
[1 2 3 4 5 6]
```

Join two arrays

In [25]:
```
import numpy as np

arr1 = np.array([1, 2, 3])

arr2 = np.array([4, 5, 6])

arr = np.concatenate((arr1, arr2))

print(arr)
```

```
[1 2 3 4 5 6]
```

Join two 2-D arrays along rows (axis=1):

In [28]:
```
arr1 = np.array([[1, 2], [3, 4]])

arr2 = np.array([[5, 6], [7, 8]])
print(arr1)
print(arr2)
```

```
[[1 2]
 [3 4]]
[[5 6]
 [7 8]]
```

In [26]:
```
import numpy as np

arr1 = np.array([[1, 2], [3, 4]])

arr2 = np.array([[5, 6], [7, 8]])

arr = np.concatenate((arr1, arr2), axis=1)
print(arr)
#If axis=0, arrays will be concatenated vertically.
#If axis=1, arrays will be concatenated  horizontally.
```

```
[[1 2 5 6]
 [3 4 7 8]]
```

In [27]:
```
import numpy as np

arr1 = np.array([[1, 2], [3, 4]])

arr2 = np.array([[5, 6], [7, 8]])

arr = np.concatenate((arr1, arr2), axis=0)

print(arr)
```

```
[[1 2]
 [3 4]
 [5 6]
 [7 8]]
```

Searching Arrays

Find the indexes where the value is 4:

In [71]:
```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 4, 4])
#    positions=[0, 1, 2, 3, 4, 5, 6]
x = np.where(arr == 4)

print(x)
```

```
(array([3, 5, 6], dtype=int64),)
```

In [72]:
```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 4, 4])
#    positions=[0, 1, 2, 3, 4, 5, 6]

x = np.where(arr == 3)

print(x)
```

```
(array([2], dtype=int64),)
```

In [73]:
```
import numpy as np

arr = np.array([1, 2, 2, 3, 4, 5, 6, 7, 10, 8])
```

```
#    positions=[0, 1, 2, 3, 4, 5, 6, 7, 8, 9 ]

x = np.where(arr%2 == 0)

print(x) #even
```

```
(array([1, 2, 4, 6, 8, 9], dtype=int64),)
```

In [74]:
```
arr = np.array([1, 2, 2, 3, 4, 5, 6, 7, 10, 8])
#    positions=[0, 1, 2, 3, 4, 5, 6, 7, 8, 9 ]

x = np.where(arr%2 != 0)

print(x) #odd
```

```
(array([0, 3, 5, 7], dtype=int64),)
```

Slicing arrays

In [75]:
```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7])
#    positions=[0, 1, 2, 3, 4, 5, 6]

print(arr[1:5]) #[1,2,3,4]
```

```
[2 3 4 5]
```

In [77]:
```
arr[3:7]#  first:last +1
```

Out[77]:
```
array([4, 5, 6, 7])
```

In [78]:
```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7])
#    positions=[0, 1, 2, 3, 4, 5, 6]

print(arr[4:])  #[4,5,6,........]
```

```
[5 6 7]
```

In [79]:
```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7])
```

```
#    positions=[0, 1, 2, 3, 4, 5, 6]

print(arr[:4])#[......3]
```

```
[1 2 3 4]
```

In [81]:
```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7])  #[0,1,2,3,4,5,6]
#    positions=[0, 1, 2, 3, 4, 5, 6]

print(arr[1:5])  #[START, STOP+1, STEP] [1,2,3,4]
```

```
[2 3 4 5]
```

In [82]:
```
print(arr[1:5:2])  #[START, STOP+1, STEP] [1,2,3,4]
```

```
[2 4]
```

In [90]:
```
arr = np.array([1, 2, 3, 4, 5, 6, 7,8 ])  #[0,1,2,3,4,5,6,7]
arr[0:7:3]
#[1,4,7]
```

Out[90]:
```
array([1, 4, 7])
```

In [99]:
```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7])

print(arr[: :-3])
```

```
[7 4 1]
```

The first part (:)

specifies the start index. In this case, it's empty, indicating that we want to start from the beginning of the array.

The second part (:)

specifies the end index. It's also empty, indicating that we want to slice until the end of the array.

The third part (-1)

specifies the step size. A negative step size means we want to step through the array in reverse order.

In [ ]:
```python
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7])

print(arr[::-2])
```

uses of an array

Data Storage: Arrays efficiently store collections of elements of the same type in contiguous memory locations.

Data Manipulation: Arrays allow for easy access, insertion, deletion, and modification of elements, facilitating data manipulation.

Mathematical Computations: Arrays are essential for mathematical computations and scientific computing, especially in libraries like NumPy.

Image Processing: Arrays represent images in image processing, enabling various operations like filtering and enhancement.

In [ ]:
```python

```

In [ ]:
```python

```